

Guía Introductoria de FastAPI

De cero a nivel intermedio: construye APIs modernas con Python

Requisitos previos: Conocimientos básicos de Python (variables, funciones, listas, diccionarios).

Objetivo: Que puedas crear APIs REST funcionales, validadas y documentadas desde el primer día.

Tabla de Contenidos

1. ¿Qué es FastAPI y por qué usarlo?
 2. Conceptos Python necesarios
 3. Instalación y configuración del entorno
 4. Tu primera API en FastAPI
 5. Endpoints: GET, POST, PUT, DELETE
 6. Parámetros: Path, Query y Body
 7. Validación de datos con Pydantic
 8. Documentación automática: Swagger y ReDoc
 9. Manejo de errores
 10. Estructura básica de proyectos
 11. Buenas prácticas iniciales
 12. Proyecto práctico: API de tareas (To-Do)
 13. Próximos pasos
-

1. ¿Qué es FastAPI y por qué usarlo?

FastAPI es un framework web moderno para construir APIs con Python 3.8+ basado en los estándares **OpenAPI** y **JSON Schema**. Fue creado por Sebastián Ramírez y publicado en 2018.

¿Por qué FastAPI?

Característica	Flask	Django REST	FastAPI
Velocidad	Media	Media	Alta
Tipado de datos	Manual	Serializers	Pydantic automático
Documentación auto	No	Parcial	Sí, completa
Async nativo	No	No	Sí
Curva de aprendizaje	Baja	Alta	Baja-Media
Validación automática	No	Parcial	Sí

Casos de uso ideales

- Microservicios y APIs REST
- Backends para aplicaciones web/móvil
- Servicios de Machine Learning (FastAPI + scikit-learn, PyTorch, etc.)
- APIs de datos en tiempo real
- Prototipos rápidos con calidad de producción

Cómo funciona FastAPI internamente

FastAPI se construye sobre dos pilares:

1. **Starlette** → Maneja el servidor web ASGI (routing, middleware, WebSockets)
2. **Pydantic** → Valida y serializa datos automáticamente con type hints

Tu código FastAPI

|



[Pydantic] ← Valida, serializa y documenta modelos de datos

|



[Starlette] ← Maneja HTTP, routing, middleware

|



[Uvicorn] ← Servidor ASGI de alto rendimiento

2. Conceptos Python necesarios

Antes de entrar en FastAPI, necesitas dominar estos conceptos de Python moderno.

2.1 Type Hints (Anotaciones de tipo)

Los type hints son la base de FastAPI. Permiten declarar el tipo esperado de variables, parámetros y valores de retorno.

```
python

# Sin type hints (Python clásico)
def saludar(nombre):
    return "Hola, " + nombre

# Con type hints (Python moderno)
def saludar(nombre: str) -> str:
    return f"Hola, {nombre}"

# Type hints en variables
edad: int = 25
precio: float = 19.99
activo: bool = True
nombres: list[str] = ["Ana", "Bob"]
datos: dict[str, int] = {"edad": 25}
```

Tipos opcionales y Union

```
python
```

```
from typing import Optional, Union

# Optional[X] es equivalente a Union[X, None]
def buscar_usuario(id: int) -> Optional[str]:
    # Puede retornar un str o None
    if id == 1:
        return "Ana"
    return None

# Union: acepta múltiples tipos
def procesar(valor: Union[int, str]) -> str:
    return str(valor)

# Python 3.10+ sintaxis moderna
def buscar(id: int) -> str | None:
    pass
```

2.2 Funciones asíncronas (async/await)

FastAPI soporta tanto funciones síncronas como asíncronas. El código async es crucial para alto rendimiento.

```
python
```



```
import asyncio
import httpx

# Función síncrona normal
def obtener_datos_sync():
    # Bloquea el hilo mientras espera
    import time
    time.sleep(2)
    return {"datos": "resultado"}

# Función asíncrona
async def obtener_datos_async():
    # NO bloquea: permite que otras tareas corran mientras espera
    await asyncio.sleep(2)
    return {"datos": "resultado"}

# Ejemplo real: llamada HTTP asíncrona
async def llamar_api_externa():
    async with httpx.AsyncClient() as cliente:
        respuesta = await cliente.get("https://api.ejemplo.com/datos")
    return respuesta.json()
```

Regla práctica:

- Usa `async def` cuando hagas I/O (base de datos, HTTP, archivos)
- Usa `def` para lógica pura de CPU

2.3 Decoradores

Los decoradores modifican el comportamiento de funciones. FastAPI los usa para registrar rutas.

python

```
# Decorador simple
def mi_decorador(func):
    def wrapper(*args, **kwargs):
        print("Antes de ejecutar")
        resultado = func(*args, **kwargs)
        print("Después de ejecutar")
        return resultado
    return wrapper

@mi_decorador
def saludar():
    print("¡Hola!")

# saludar() imprimirá:
# Antes de ejecutar
# ¡Hola!
# Después de ejecutar
```

2.4 Dataclasses y clases en Python

```
python

from dataclasses import dataclass
from typing import Optional

@dataclass
class Usuario:
    nombre: str
    edad: int
    email: Optional[str] = None

    def es_mayor(self) -> bool:
        return self.edad >= 18

# Uso
user = Usuario(nombre="Ana", edad=25, email="ana@email.com")
print(user.es_mayor()) # True
```

2.5 Gestores de contexto (context managers)

```
python
```

```
# Patrón with - garantiza apertura/cierre de recursos
with open("archivo.txt", "r") as f:
    contenido = f.read()
# El archivo se cierra automáticamente al salir del bloque

# Async context manager (para conexiones de BD, HTTP clients, etc.)
async def ejemplo():
    async with httpx.AsyncClient() as cliente:
        respuesta = await cliente.get("https://api.com")
    # Cliente se cierra automáticamente
```

2.6 F-strings y formateo moderno

```
python

nombre = "Ana"
edad = 25

# F-strings (Python 3.6+) - la forma más limpia
mensaje = f"Hola {nombre}, tienes {edad} años"
url = f"https://api.ejemplo.com/usuarios/{nombre.lower()}"

# Formateo con expresiones
precio = 19.999
display = f"Precio: ${precio:.2f}" # "Precio: $20.00"
```

3. Instalación y configuración del entorno

3.1 Entorno virtual (obligatorio)

Siempre trabaja con entornos virtuales para aislar dependencias por proyecto.

```
bash
```

```
# Crear entorno virtual
python -m venv venv

# Activar (Linux/macOS)
source venv/bin/activate

# Activar (Windows)
venv\Scripts\activate

# Verificar que está activo (verás el prefijo (venv) en el terminal)
which python # Linux/macOS
where python # Windows
```

3.2 Instalación de FastAPI

```
bash

# Instalación básica (FastAPI + Uvicorn)
pip install fastapi uvicorn

# Instalación completa con extras recomendados
pip install fastapi[all]
# Incluye: uvicorn, pydantic, email-validator, python-multipart, etc.

# Instalar también herramientas de desarrollo
pip install httpx pytest pytest-asyncio

# Guardar dependencias
pip freeze > requirements.txt
```

3.3 Estructura mínima de archivos

```
mi_proyecto/
├── venv/           # Entorno virtual (nunca en git)
├── main.py         # Punto de entrada
├── requirements.txt # Dependencias
└── .gitignore      # main.py incluye: venv/, __pycache__/
```

3.4 Verificar la instalación

```
python
```

```
# main.py
import fastapi
import uvicorn

print(f"FastAPI versión: {fastapi.__version__}")
```

```
bash

python main.py
# FastAPI versión: 0.110.x (o la versión instalada)
```

3.5 Editores recomendados

Editor	Por qué usarlo con FastAPI
VS Code	Extensión Pylance para type hints, debugging integrado
PyCharm	Soporte completo, inspección de tipos avanzada
Neovim/Vim	Para usuarios avanzados con LSP configurado

Extensiones recomendadas para VS Code:

- Python (Microsoft)
- Pylance
- REST Client (para probar APIs sin Swagger)
- Thunder Client (alternativa a Postman integrada)

4. Tu primera API en FastAPI

4.1 Hola Mundo

```
python
```

```
# main.py
from fastapi import FastAPI

# Crear instancia de la aplicación
app = FastAPI(
    title="Mi Primera API",
    description="API de demostración con FastAPI",
    version="1.0.0"
)

# Definir un endpoint GET en la ruta raíz "/"
@app.get("/")
def inicio():
    return {"mensaje": "¡Hola desde FastAPI!"}

@app.get("/salud")
def verificar_salud():
    return {"estado": "OK", "servicio": "Mi API"}
```

4.2 Ejecutar el servidor

```
bash

# Modo desarrollo con recarga automática
uvicorn main:app --reload

# Especificar host y puerto
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Verás en la consola:

```
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process
INFO:      Application startup complete.
```

4.3 Probar la API

```
bash
```

```
# Con curl
curl http://localhost:8000/
# {"mensaje":"¡Hola desde FastAPI!"}

# Con curl más legible
curl -s http://localhost:8000/ | python -m json.tool
```

O abre el navegador en:

- [**http://localhost:8000/**](http://localhost:8000/) → Tu endpoint
- [**http://localhost:8000/docs**](http://localhost:8000/docs) → Swagger UI (documentación interactiva)
- [**http://localhost:8000/redoc**](http://localhost:8000/redoc) → ReDoc (documentación alternativa)

4.4 Entender el objeto FastAPI

```
python
```

```
from fastapi import FastAPI

app = FastAPI(
    # Información de la API (aparece en la documentación)
    title="API de Usuarios",
    description="""
    ## API para gestión de usuarios

    Permite crear, leer, actualizar y eliminar usuarios.

    ### Funcionalidades
    - Autenticación con JWT
    - Validación de datos automática
    """,
    version="2.1.0",

    # URLs personalizadas para la documentación
    docs_url="/documentacion",    # Swagger en /documentacion
    redoc_url="/redoc",

    # Información de contacto (opcional)
    contact={
        "name": "Equipo de Desarrollo",
        "email": "dev@empresa.com"
    },

    # Licencia
    license_info={
        "name": "MIT",
        "url": "https://opensource.org/licenses/MIT"
    }
)
```

5. Endpoints: GET, POST, PUT, DELETE

5.1 Método GET — Obtener datos

```
python
```



```
from fastapi import FastAPI
from typing import Optional

app = FastAPI()

# Base de datos simulada (en memoria)
usuarios_db = [
    {"id": 1, "nombre": "Ana García", "email": "ana@email.com", "activo": True},
    {"id": 2, "nombre": "Bob Martínez", "email": "bob@email.com", "activo": False},
    {"id": 3, "nombre": "Carol López", "email": "carol@email.com", "activo": True},
]

@app.get("/usuarios")
def listar_usuarios():
    """Retorna todos los usuarios."""
    return usuarios_db

@app.get("/usuarios/{id}")
def obtener_usuario(id: int):
    """Retorna un usuario por su ID."""
    for usuario in usuarios_db:
        if usuario["id"] == id:
            return usuario
    return {"error": "Usuario no encontrado"}
```

5.2 Método POST — Crear recursos

python

```

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class UsuarioCrear(BaseModel):
    nombre: str
    email: str
    activo: bool = True # Valor por defecto

usuarios_db = []
contador_id = 1

@app.post("/usuarios", status_code=201) # 201 = Created
def crear_usuario(usuario: UsuarioCrear):
    """Crea un nuevo usuario."""
    global contador_id

    nuevo_usuario = {
        "id": contador_id,
        "nombre": usuario.nombre,
        "email": usuario.email,
        "activo": usuario.activo
    }

    usuarios_db.append(nuevo_usuario)
    contador_id += 1

    return nuevo_usuario

```

Probar con curl:

```

bash

curl -X POST http://localhost:8000/usuarios \
  -H "Content-Type: application/json" \
  -d '{"nombre": "Diana", "email": "diana@email.com"}'

```

5.3 Método PUT — Actualizar completamente

```
python
```

```

@app.put("/usuarios/{id}")
def actualizar_usuario(id: int, usuario: UsuarioCrear):
    """Reemplaza completamente un usuario."""
    for index, u in enumerate(usuarios_db):
        if u["id"] == id:
            usuarios_db[index] = {
                "id": id,
                "nombre": usuario.nombre,
                "email": usuario.email,
                "activo": usuario.activo
            }
            return usuarios_db[index]
    return {"error": "Usuario no encontrado"}

```

5.4 Método PATCH — Actualización parcial

```

python

from typing import Optional

class UsuarioActualizar(BaseModel):
    nombre: Optional[str] = None
    email: Optional[str] = None
    activo: Optional[bool] = None

@app.patch("/usuarios/{id}")
def actualizar_parcial(id: int, datos: UsuarioActualizar):
    """Actualiza solo los campos proporcionados."""
    for index, u in enumerate(usuarios_db):
        if u["id"] == id:
            # Solo actualiza campos que no son None
            if datos.nombre is not None:
                usuarios_db[index]["nombre"] = datos.nombre
            if datos.email is not None:
                usuarios_db[index]["email"] = datos.email
            if datos.activo is not None:
                usuarios_db[index]["activo"] = datos.activo
            return usuarios_db[index]
    return {"error": "Usuario no encontrado"}

```

5.5 Método DELETE — Eliminar recursos

```
python
```

```

from fastapi import FastAPI, Response, status

@app.delete("/usuarios/{id}", status_code=204)
def eliminar_usuario(id: int, response: Response):
    """Elimina un usuario por ID."""
    for index, u in enumerate(usuarios_db):
        if u["id"] == id:
            usuarios_db.pop(index)
            return # 204 No Content: no retorna cuerpo

    # Si no existe, cambiar el código de estado
    response.status_code = status.HTTP_404_NOT_FOUND
    return {"error": "Usuario no encontrado"}

```

5.6 Response models y códigos de estado

```

python

from fastapi import FastAPI, status
from pydantic import BaseModel
from typing import List

class UsuarioResponse(BaseModel):
    id: int
    nombre: str
    email: str
    # Nota: no incluimos campos internos como contraseña

# El response_model filtra automáticamente los campos
@app.get(
    "/usuarios",
    response_model=List[UsuarioResponse],
    status_code=status.HTTP_200_OK,
    summary="Listar usuarios",
    description="Retorna todos los usuarios activos del sistema",
    tags=["usuarios"] # Agrupa en la documentación
)
def listar_usuarios():
    return usuarios_db

```

6. Parámetros: Path, Query y Body

6.1 Parámetros de ruta (Path Parameters)

Están en la URL entre llaves `{parametro}`.

```
python

from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/usuarios/{usuario_id}")
def obtener_usuario(
    usuario_id: int = Path(
        ..., # ... significa obligatorio
        title="ID del Usuario",
        description="El identificador único del usuario",
        gt=0, # greater than 0
        le=1000 # less or equal to 1000
    )
):
    return {"id": usuario_id}

@app.get("/archivos/{ruta_completa:path}")
def obtener_archivo(ruta_completa: str):
    # Captura rutas con slash: /archivos/carpeta/subcarpeta/archivo.txt
    return {"ruta": ruta_completa}

@app.get("/items/{item_id}/comentarios/{comentario_id}")
def obtener_comentario(item_id: int, comentario_id: int):
    # Múltiples parámetros en la misma ruta
    return {"item": item_id, "comentario": comentario_id}
```

6.2 Parámetros de consulta (Query Parameters)

Van después del `?` en la URL: `/usuarios?pagina=2&limite=10`.

```
python
```

```

from fastapi import FastAPI, Query
from typing import Optional, List

app = FastAPI()

@app.get("/usuarios")
def listar_usuarios(
    # Parámetros opcionales con valores por defecto
    pagina: int = Query(default=1, ge=1, description="Número de página"),
    limite: int = Query(default=10, ge=1, le=100, description="Items por página"),
    activo: Optional[bool] = Query(default=None, description="Filtrar por estado"),
    buscar: Optional[str] = Query(default=None, min_length=3, max_length=50)
):
    """
    Lista usuarios con paginación y filtros.

    URL ejemplo: /usuarios?pagina=2&limite=5&activo=true&buscar=ana
    """
    return {
        "pagina": pagina,
        "limite": limite,
        "filtros": {
            "activo": activo,
            "buscar": buscar
        }
    }

# Query con múltiples valores del mismo parámetro
@app.get("/items")
def listar_items(
    tags: List[str] = Query(default=[], description="Filtrar por tags")
):
    # URL: /items?tags=python&tags=fastapi&tags=api
    return {"tags_filtrados": tags}

```

6.3 Cuerpo de la petición (Request Body)

python

```

from fastapi import FastAPI, Body
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

class Producto(BaseModel):
    nombre: str
    descripcion: Optional[str] = None
    precio: float
    stock: int = 0

class Proveedor(BaseModel):
    nombre: str
    contacto: str

# Body simple
@app.post("/productos")
def crear_producto(producto: Producto):
    return {"creado": producto}

# Múltiples bodies
@app.post("/productos/con-proveedor")
def crear_con_proveedor(
    producto: Producto,
    proveedor: Proveedor,
    notas: str = Body(default="Sin notas", embed=True)
):
    return {
        "producto": producto,
        "proveedor": proveedor,
        "notas": notas
    }

```

JSON esperado para múltiples bodies:

```

json

```

```
{
  "producto": {
    "nombre": "Laptop",
    "precio": 999.99,
    "stock": 10
  },
  "proveedor": {
    "nombre": "TechCorp",
    "contacto": "info@techcorp.com"
  },
  "notas": "Entrega en 5 días"
}
```

6.4 Headers y Cookies

```
python

from fastapi import FastAPI, Header, Cookie
from typing import Optional

app = FastAPI()

@app.get("/perfil")
def obtener_perfil(
    # Headers (FastAPI convierte guiones a guiones_bajos)
    user_agent: Optional[str] = Header(default=None),
    x_token: Optional[str] = Header(default=None), # X-Token header

    # Cookies
    session_id: Optional[str] = Cookie(default=None)
):
    return {
        "user_agent": user_agent,
        "token": x_token,
        "session": session_id
    }
```

6.5 Combinando todos los parámetros

```
python
```



```
@app.put("/tiendas/{tienda_id}/productos/{producto_id}")
def actualizar_producto_tienda(
    # Path parameters
    tienda_id: int = Path(..., gt=0),
    producto_id: int = Path(..., gt=0),

    # Query parameters
    notificar: bool = Query(default=False),

    # Body
    producto: Producto = Body(...),

    # Header
    x_admin_key: str = Header(...)
):
    return {
        "tienda": tienda_id,
        "producto": producto_id,
        "datos": producto,
        "notificar": notificar
    }
```

7. Validación de datos con Pydantic

Pydantic es el motor de validación de FastAPI. Garantiza que los datos sean correctos antes de llegar a tu lógica.

7.1 Modelos básicos

```
python
```

```

from pydantic import BaseModel, Field
from typing import Optional, List
from datetime import datetime

class Direccion(BaseModel):
    calle: str
    ciudad: str
    pais: str = "Guatemala"
    codigo_postal: Optional[str] = None

class Usuario(BaseModel):
    # Campo obligatorio
    nombre: str

    # Campo obligatorio con validación
    email: str

    # Campo opcional con valor por defecto
    edad: Optional[int] = None

    # Campo con metadatos (aparecen en la documentación)
    descripcion: str = Field(
        default="",
        title="Descripción del usuario",
        description="Descripción pública del perfil",
        max_length=500,
        example="Desarrollador apasionado por Python"
    )

    # Relaciones (modelos anidados)
    direccion: Optional[Direccion] = None

    # Listas
    tags: List[str] = []

    # Fecha automática
    creado_en: datetime = Field(default_factory=datetime.now)

```

7.2 Validaciones con Field

python

```

from pydantic import BaseModel, Field

class Producto(BaseModel):
    nombre: str = Field(
        ...,
        min_length=2,
        max_length=100,
        example="Laptop Pro"
    )

    precio: float = Field(
        ...,
        gt=0,          # mayor que 0
        le=999999,     # menor o igual a 999999
        example=999.99
    )

    stock: int = Field(
        default=0,
        ge=0,          # mayor o igual a 0
        le=10000
    )

    codigo_sku: str = Field(
        ...,
        pattern=r"^[A-Z]{3}-\d{4}$", # Regex: ABC-1234
        example="LAP-0001"
    )

    descuento: float = Field(
        default=0.0,
        ge=0.0,
        le=1.0,        # Porcentaje entre 0 y 1
        description="Descuento como decimal: 0.10 = 10%"
    )

```

7.3 Validadores personalizados

python

```
from pydantic import BaseModel, field_validator, model_validator
from typing import Optional

class UsuarioRegistro(BaseModel):
    username: str
    email: str
    password: str
    confirmar_password: str
    edad: int

    # Validador de campo individual
    @field_validator("username")
    @classmethod
    def username_debe_ser_valido(cls, v: str) -> str:
        if not v.isalnum():
            raise ValueError("El username solo puede contener letras y números")
        if len(v) < 3:
            raise ValueError("El username debe tener al menos 3 caracteres")
        return v.lower() # Normalizar a minúsculas

    @field_validator("email")
    @classmethod
    def email_debe_ser_valido(cls, v: str) -> str:
        if "@" not in v or "." not in v.split("@")[-1]:
            raise ValueError("Email inválido")
        return v.lower()

    @field_validator("edad")
    @classmethod
    def edad_debe_ser_mayor(cls, v: int) -> int:
        if v < 18:
            raise ValueError("Debes ser mayor de 18 años")
        if v > 120:
            raise ValueError("Edad inválida")
        return v

    # Validador que accede a múltiples campos
    @model_validator(mode="after")
    def passwords_deben_coincidir(self) -> "UsuarioRegistro":
        if self.password != self.confirmar_password:
            raise ValueError("Las contraseñas no coinciden")
        return self
```

7.4 Configuración del modelo

python

```
from pydantic import BaseModel, ConfigDict

class Usuario(BaseModel):
    model_config = ConfigDict(
        # Permite crear el modelo desde objetos ORM
        from_attributes=True,

        # Ignora campos extras en el JSON (no lanza error)
        extra="ignore",

        # Valida también en asignaciones: user.nombre = 123 lanzará error
        validate_assignment=True,

        # Hace inmutable el modelo
        frozen=False,

        # Schema con ejemplos
        json_schema_extra={
            "example": {
                "nombre": "Ana García",
                "email": "ana@email.com"
            }
        }
    )

    nombre: str
    email: str
```

7.5 Serialización y deserialización

python

```
from pydantic import BaseModel
import json

class Producto(BaseModel):
    nombre: str
    precio: float
    activo: bool = True

# Crear desde dict
p = Producto(**{"nombre": "Laptop", "precio": 999.99})
p = Producto.model_validate({"nombre": "Laptop", "precio": 999.99})

# Crear desde JSON string
p = Producto.model_validate_json('{"nombre": "Laptop", "precio": 999.99}')

# Serializar a dict
d = p.model_dump() # {"nombre": "Laptop", "precio": 999.99, "activo": True}
d = p.model_dump(exclude={"activo"}) # Excluir campos
d = p.model_dump(include={"nombre"}) # Solo incluir estos campos

# Serializar a JSON
j = p.model_dump_json()
j = p.model_dump_json(indent=2) # Pretty print
```

7.6 Herencia de modelos (patrón Create/Read/Update)

python

```

from pydantic import BaseModel
from typing import Optional
from datetime import datetime

# Modelo base con campos comunes
class UsuarioBase(BaseModel):
    nombre: str
    email: str

# Para crear (sin ID, sin fecha de creación)
class UsuarioCrear(UsuarioBase):
    password: str

# Para leer (con ID y fecha)
class UsuarioLeer(UsuarioBase):
    id: int
    creado_en: datetime
    activo: bool

    model_config = ConfigDict(from_attributes=True)

# Para actualizar (todos opcionales)
class UsuarioActualizar(BaseModel):
    nombre: Optional[str] = None
    email: Optional[str] = None

# Uso en endpoints
@app.post("/usuarios", response_model=UsuarioLeer, status_code=201)
def crear_usuario(usuario: UsuarioCrear):
    # FastAPI usa UsuarioCrear para validar la entrada
    # y UsuarioLeer para filtrar la salida (nunca expone la password)
    nuevo = guardar_en_db(usuario)
    return nuevo

```

8. Documentación automática: Swagger y ReDoc

Una de las mayores ventajas de FastAPI es la generación automática de documentación.

8.1 Swagger UI

Accede en <http://localhost:8000/docs>

Swagger UI te permite:

- Ver todos los endpoints organizados por tags
- Probar cada endpoint directamente desde el navegador
- Ver los schemas de request y response
- Autenticarte y probar endpoints protegidos

8.2 Mejorar la documentación

```
python
```



```

from fastapi import FastAPI
from pydantic import BaseModel, Field
from typing import List

app = FastAPI(
    title="API de E-commerce",
    description=""
## API de E-commerce

Gestiona productos, usuarios y pedidos.

### Autenticación
Todos los endpoints de `/admin/*` requieren JWT token.

### Paginación
Los endpoints de listado aceptan `pagina` y `limite`.
    """
    version="1.0.0",
    openapi_tags=[
        {"name": "usuarios", "description": "Gestión de usuarios y perfiles"},
        {"name": "productos", "description": "Catálogo de productos"},
        {"name": "pedidos", "description": "Gestión de pedidos"},
    ]
)

class ProductoCrear(BaseModel):
    nombre: str = Field(..., example="Laptop Pro 15")
    precio: float = Field(..., example=1299.99)

    model_config = {
        "json_schema_extra": {
            "examples": [
                {
                    "nombre": "Laptop Pro",
                    "precio": 999.99
                },
                {
                    "nombre": "Mouse Inalámbrico",
                    "precio": 29.99
                }
            ]
        }
    }

```

```

@app.post(
    "/productos",
    response_model=dict,
    status_code=201,
    tags=["productos"],
    summary="Crear producto",
    description=""
    Crea un nuevo producto en el catálogo.

    **Reglas:**
    - El nombre debe ser único
    - El precio debe ser mayor a 0
    - Solo usuarios admin pueden usar este endpoint
    """
    response_description="El producto creado con su ID asignado",
    deprecated=False # Marcar endpoint como obsoleto cuando sea necesario
)
def crear_producto(producto: ProductoCrear):
    return {"id": 1, **producto.model_dump()}

```

8.3 OpenAPI schema personalizado

python

```
from fastapi import FastAPI
from fastapi.openapi.utils import get_openapi

app = FastAPI()

def schema_personalizado():
    if app.openapi_schema:
        return app.openapi_schema

    openapi_schema = get_openapi(
        title="Mi API",
        version="1.0.0",
        description="API personalizada",
        routes=app.routes,
    )

    # Agregar información de seguridad
    openapi_schema["info"]["x-logo"] = {
        "url": "https://mi-empresa.com/logo.png"
    }

    app.openapi_schema = openapi_schema
    return app.openapi_schema

app.openapi = schema_personalizado
```

9. Manejo de errores

9.1 HTTPException básico

```
python
```

```
from fastapi import FastAPI, HTTPException, status

app = FastAPI()

@app.get("/usuarios/{id}")
def obtener_usuario(id: int):
    usuario = buscar_en_db(id)

    if usuario is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Usuario con ID {id} no encontrado"
        )

    return usuario

@app.delete("/usuarios/{id}")
def eliminar_usuario(id: int, es_admin: bool = False):
    if not es_admin:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="No tienes permisos para eliminar usuarios",
            headers={"X-Error": "Insufficient-Permissions"}
        )

    # Eliminar usuario...
    return {"mensaje": "Usuario eliminado"}
```

9.2 Códigos de estado HTTP más usados

python

```
from fastapi import status
```

```
# 2xx - Éxito
```

```
status.HTTP_200_OK           # GET exitoso  
status.HTTP_201_CREATED      # POST exitoso (recurso creado)  
status.HTTP_204_NO_CONTENT   # DELETE exitoso (sin cuerpo)
```

```
# 4xx - Error del cliente
```

```
status.HTTP_400_BAD_REQUEST  # Datos inválidos  
status.HTTP_401_UNAUTHORIZED # No autenticado  
status.HTTP_403_FORBIDDEN    # Autenticado pero sin permisos  
status.HTTP_404_NOT_FOUND     # Recurso no encontrado  
status.HTTP_409_CONFLICT      # Conflicto (ej: email duplicado)  
status.HTTP_422_UNPROCESSABLE_ENTITY # Error de validación (Pydantic)
```

```
# 5xx - Error del servidor
```

```
status.HTTP_500_INTERNAL_SERVER_ERROR # Error inesperado  
status.HTTP_503_SERVICE_UNAVAILABLE   # Servicio no disponible
```

9.3 Excepciones personalizadas

```
python
```

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()

# Definir excepción personalizada
class UsuarioNoEncontradoError(Exception):
    def __init__(self, usuario_id: int):
        self.usuario_id = usuario_id

class EmailDuplicadoError(Exception):
    def __init__(self, email: str):
        self.email = email

# Registrar manejadores de excepción
@app.exception_handler(UsuarioNoEncontradoError)
async def manejar_usuario_no_encontrado(
    request: Request,
    exc: UsuarioNoEncontradoError
):
    return JSONResponse(
        status_code=404,
        content={
            "error": "usuario_no_encontrado",
            "mensaje": f"No existe el usuario con ID {exc.usuario_id}",
            "id_buscado": exc.usuario_id
        }
    )

@app.exception_handler(EmailDuplicadoError)
async def manejar_email_duplicado(request: Request, exc: EmailDuplicadoError):
    return JSONResponse(
        status_code=409,
        content={
            "error": "email_duplicado",
            "mensaje": f"El email {exc.email} ya está registrado"
        }
    )

# Uso en endpoints
@app.get("/usuarios/{id}")
def obtener_usuario(id: int):
    usuario = buscar_en_db(id)
```

```
    if not usuario:
        raise UsuarioNoEncontradoError(usuario_id=id)
    return usuario

@app.post("/usuarios")
def crear_usuario(usuario: dict):
    if email_existe(usuario["email"]):
        raise EmailDuplicadoError(email=usuario["email"])
    return guardar_usuario(usuario)
```

9.4 Manejador global de errores

python

```

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError
from pydantic import ValidationError

app = FastAPI()

# Sobrescribir el manejador de errores de validación (422)
@app.exception_handler(RequestValidationError)
async def manejar_error_validacion(request: Request, exc: RequestValidationError):
    errores = []
    for error in exc.errors():
        errores.append({
            "campo": " → ".join(str(loc) for loc in error["loc"]),
            "mensaje": error["msg"],
            "tipo": error["type"]
        })

    return JSONResponse(
        status_code=422,
        content={
            "error": "datos_invalidos",
            "mensaje": "Los datos enviados no son válidos",
            "detalles": errores
        }
    )

# Manejador para errores 500 (no dejes que el stack trace llegue al cliente)
@app.exception_handler(Exception)
async def manejar_error_interno(request: Request, exc: Exception):
    # Aquí deberías loggear el error real
    print(f"Error interno: {exc}") # En producción usa logging

    return JSONResponse(
        status_code=500,
        content={
            "error": "error_interno",
            "mensaje": "Ha ocurrido un error interno. Por favor intenta más tarde."
        }
    )

```


9.5 Respuestas de error estandarizadas

python

```
from pydantic import BaseModel
from typing import Optional, List, Any

class ErrorDetalle(BaseModel):
    campo: Optional[str] = None
    mensaje: str

class RespuestaError(BaseModel):
    exito: bool = False
    codigo_error: str
    mensaje: str
    detalles: Optional[List[ErrorDetalle]] = None

class RespuestaExito(BaseModel):
    exito: bool = True
    datos: Any
    mensaje: Optional[str] = None

# Uso
@app.get(
    "/usuarios/{id}",
    responses={
        200: {"model": RespuestaExito},
        404: {"model": RespuestaError},
        500: {"model": RespuestaError}
    }
)
def obtener_usuario(id: int):
    usuario = buscar_en_db(id)
    if not usuario:
        raise HTTPException(
            status_code=404,
            detail=RespuestaError(
                codigo_error="usuario_no_encontrado",
                mensaje=f"No existe usuario con ID {id}"
            ).model_dump()
        )
    return RespuestaExito(datos=usuario)
```

10. Estructura básica de proyectos

10.1 Proyecto pequeño (1-3 módulos)

```
mi_api/  
├─ main.py          # App FastAPI y configuración  
├─ modelos.py       # Modelos Pydantic  
├─ rutas.py         # Endpoints  
├─ requirements.txt  
└─ .env             # Variables de entorno (nunca en git)
```

python

```
# modelos.py  
from pydantic import BaseModel  
from typing import Optional  
  
class UsuarioBase(BaseModel):  
    nombre: str  
    email: str  
  
class UsuarioCrear(UsuarioBase):  
    password: str  
  
class UsuarioLeer(UsuarioBase):  
    id: int
```

python

```
# rutas.py  
from fastapi import APIRouter, HTTPException  
from .modelos import UsuarioCrear, UsuarioLeer  
  
router = APIRouter(prefix="/usuarios", tags=["usuarios"])  
  
@router.get("/", response_model=list[UsuarioLeer])  
def listar_usuarios():  
    return []  
  
@router.post("/", response_model=UsuarioLeer, status_code=201)  
def crear_usuario(usuario: UsuarioCrear):  
    return {"id": 1, **usuario.model_dump(exclude={"password"})}
```

```
python
```

```
# main.py
from fastapi import FastAPI
from .rutas import router as usuarios_router

app = FastAPI(title="Mi API")
app.include_router(usuarios_router)
```

10.2 Proyecto mediano (estructura recomendada)

```
mi_api/
├─ app/
│   ├── __init__.py
│   ├── main.py           # Punto de entrada, configura la app
│   ├── config.py        # Configuración y variables de entorno
│   └─ modelos/
│       ├── __init__.py
│       ├── usuario.py    # Modelos Pydantic de usuario
│       └─ producto.py
│   └─ rutas/
│       ├── __init__.py
│       ├── usuarios.py   # Router de usuarios
│       └─ productos.py
│   └─ servicios/
│       ├── __init__.py
│       └─ usuario_servicio.py # Lógica de negocio
│   └─ utilidades/
│       ├── __init__.py
│       └─ seguridad.py
├─ tests/
│   ├── conftest.py
│   ├── test_usuarios.py
│   └─ test_productos.py
├─ .env
├─ .env.example
├─ requirements.txt
├─ requirements-dev.txt
└─ README.md
```

10.3 APIRouter para organizar endpoints

```
python
```

```
# app/rutas/usuarios.py
from fastapi import APIRouter, HTTPException, Depends, status
from app.modelos.usuario import UsuarioCrear, UsuarioLeer, UsuarioActualizar
from app.servicios.usuario_servicio import UsuarioServicio

# Router con prefijo y tag automáticos
router = APIRouter(
    prefix="/usuarios",
    tags=["usuarios"],
    responses={404: {"description": "No encontrado"}}
)

@router.get("/", response_model=list[UsuarioLeer])
async def listar_usuarios(
    pagina: int = 1,
    limite: int = 10,
    servicio: UsuarioServicio = Depends()
):
    return await servicio.listar(pagina, limite)

@router.get("/{usuario_id}", response_model=UsuarioLeer)
async def obtener_usuario(usuario_id: int):
    pass

@router.post("/", response_model=UsuarioLeer, status_code=status.HTTP_201_CREATED)
async def crear_usuario(usuario: UsuarioCrear):
    pass
```

python

```
# app/main.py
from fastapi import FastAPI
from app.rutas import usuarios, productos, pedidos

app = FastAPI()

# Registrar todos los routers
app.include_router(usuarios.router)
app.include_router(productos.router)
app.include_router(pedidos.router)

# También puedes agregar prefijos globales
app.include_router(usuarios.router, prefix="/api/v1")
```

10.4 Configuración con variables de entorno

```
python
```

```
# app/config.py
from pydantic_settings import BaseSettings
from functools import lru_cache

class Configuracion(BaseSettings):
    # Nombre de la aplicación
    nombre_app: str = "Mi API"
    version: str = "1.0.0"

    # Entorno
    entorno: str = "desarrollo" # desarrollo, staging, produccion
    debug: bool = True

    # Base de datos
    database_url: str = "sqlite:///./app.db"

    # Seguridad
    secret_key: str = "clave-secreta-cambiar-en-produccion"
    token_expiracion_minutos: int = 30

    # CORS
    origenes_permitidos: list[str] = ["http://localhost:3000"]

    class Config:
        env_file = ".env" # Lee el archivo .env automáticamente

# Singleton con caché
@lru_cache()
def obtener_config() -> Configuracion:
    return Configuracion()

# Uso en cualquier parte del proyecto
config = obtener_config()
print(config.database_url)
```

bash

```
# .env
NOMBRE_APP=Mí API Producción
ENTORNO=producción
DEBUG=false
DATABASE_URL=postgresql://usuario:password@localhost/mi_db
SECRET_KEY=super-secreto-y-largo-key-aquí
```

11. Buenas prácticas iniciales

11.1 Nomenclatura y estilo

```
python

# ✅ CORRECTO: snake_case para variables y funciones
def obtener_usuario_por_email(email: str) -> dict:
    pass

# ✅ CORRECTO: PascalCase para clases y modelos
class UsuarioServicio:
    pass

class ModeloUsuario(BaseModel):
    pass

# ✅ CORRECTO: Nombres descriptivos
@app.get("/usuarios/{usuario_id}/pedidos/activos")
def obtener_pedidos_activos_de_usuario(usuario_id: int):
    pass

# ❌ INCORRECTO: Nombres vagos
@app.get("/u/{id}/p/a")
def get(id):
    pass
```

11.2 Principio de responsabilidad única

```
python
```

```

# ❌ MAL: Todo en el endpoint
@app.post("/usuarios")
async def crear_usuario(usuario: UsuarioCrear):
    # Validar email
    if "@" not in usuario.email:
        raise HTTPException(400, "Email inválido")

    # Verificar duplicado
    existe = db.query(Usuario).filter(Usuario.email == usuario.email).first()
    if existe:
        raise HTTPException(409, "Email duplicado")

    # Hash de contraseña
    hash_password = bcrypt.hash(usuario.password)

    # Guardar
    nuevo = Usuario(nombre=usuario.nombre, email=usuario.email, password=hash_password)
    db.add(nuevo)
    db.commit()

    # Enviar email
    enviar_email_bienvenida(usuario.email)

    return nuevo

# ✅ BIEN: Separar responsabilidades
@app.post("/usuarios")
async def crear_usuario(
    usuario: UsuarioCrear,
    servicio: UsuarioServicio = Depends()
):
    return await servicio.crear(usuario) # Toda la lógica en el servicio

```

11.3 Tipado estricto siempre

python


```
# ❌ Sin tipos
def calcular_total(items):
    return sum(item["precio"] * item["cantidad"] for item in items)

# ✅ Con tipos
from typing import TypedDict, List

class ItemPedido(TypedDict):
    precio: float
    cantidad: int

def calcular_total(items: List[ItemPedido]) -> float:
    return sum(item["precio"] * item["cantidad"] for item in items)
```

11.4 Documentar siempre

```
python

@app.get(
    "/productos/{producto_id}",
    summary="Obtener producto por ID",
    response_model=ProductoLeer,
    responses={
        200: {"description": "Producto encontrado"},
        404: {"description": "Producto no existe"}
    }
)
async def obtener_producto(producto_id: int):
    """
    Retorna los detalles de un producto específico.

    - **producto_id**: El ID único del producto (entero positivo)

    Lanza 404 si el producto no existe en la base de datos.
    """
    pass
```

11.5 No expongas datos sensibles

```
python
```

```
# ❌ MAL: Expone la contraseña
class Usuario(BaseModel):
    id: int
    nombre: str
    email: str
    password: str # ¡NUNCA exponer!

# ✅ BIEN: Modelo separado para respuesta
class UsuarioCrear(BaseModel):
    nombre: str
    email: str
    password: str # Solo para recibir

class UsuarioRespuesta(BaseModel):
    id: int
    nombre: str
    email: str
    # Sin password

@app.post("/usuarios", response_model=UsuarioRespuesta)
def crear_usuario(usuario: UsuarioCrear):
    # FastAPI filtra automáticamente con response_model
    pass
```

11.6 Usa async cuando corresponde

python

```
# ✅ Usa async para operaciones I/O (BD, HTTP, archivos)
@app.get("/usuarios")
async def listar_usuarios(db: AsyncSession = Depends(obtener_db)):
    resultado = await db.execute(select(Usuario))
    return resultado.scalars().all()

# ✅ Usa def para lógica pura (sin I/O)
@app.get("/calcular")
def calcular_fibonacci(n: int):
    # Lógica de CPU, no I/O
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

12. Proyecto práctico: API de tareas (To-Do)

Vamos a construir una API completa de gestión de tareas aplicando todo lo aprendido.

12.1 Estructura del proyecto

```
todo_api/
├── main.py
├── modelos.py
├── datos.py          # "Base de datos" en memoria
└── requirements.txt
```

12.2 Modelos (modelos.py)

```
python
```

```
# modelos.py
from pydantic import BaseModel, Field, field_validator
from typing import Optional
from datetime import datetime
from enum import Enum

class PrioridadTarea(str, Enum):
    BAJA = "baja"
    MEDIA = "media"
    ALTA = "alta"
    CRITICA = "critica"

class EstadoTarea(str, Enum):
    PENDIENTE = "pendiente"
    EN_PROGRESO = "en_progreso"
    COMPLETADA = "completada"
    CANCELADA = "cancelada"

class TareaBase(BaseModel):
    titulo: str = Field(
        ...,
        min_length=3,
        max_length=200,
        example="Revisar documentación de FastAPI"
    )
    descripcion: Optional[str] = Field(
        default=None,
        max_length=1000
    )
    prioridad: PrioridadTarea = PrioridadTarea.MEDIA
    fecha_vencimiento: Optional[datetime] = None

    @field_validator("titulo")
    @classmethod
    def titulo_no_puede_ser_vacio(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("El título no puede contener solo espacios")
        return v.strip()

class TareaCrear(TareaBase):
    pass

class TareaActualizar(BaseModel):
```

```
titulo: Optional[str] = Field(default=None, min_length=3, max_length=200)
descripcion: Optional[str] = None
prioridad: Optional[PrioridadTarea] = None
estado: Optional[EstadoTarea] = None
fecha_vencimiento: Optional[datetime] = None
```

```
class TareaLeer(TareaBase):
    id: int
    estado: EstadoTarea
    creada_en: datetime
    actualizada_en: datetime

    model_config = {"from_attributes": True}
```

12.3 Datos en memoria (datos.py)

python

```

# datos.py
from datetime import datetime
from modelos import TareaLeer, EstadoTarea, PrioridadTarea

# Simulación de base de datos
_tareas: dict[int, dict] = {}
_contador: int = 0

def obtener_siguiete_id() -> int:
    global _contador
    _contador += 1
    return _contador

def crear_tarea(datos: dict) -> dict:
    ahora = datetime.now()
    tarea = {
        "id": obtener_siguiete_id(),
        "estado": EstadoTarea.PENDIENTE,
        "creada_en": ahora,
        "actualizada_en": ahora,
        **datos
    }
    _tareas[tarea["id"]] = tarea
    return tarea

def obtener_todas(
    estado: str | None = None,
    prioridad: str | None = None,
    pagina: int = 1,
    limite: int = 10
) -> list[dict]:
    tareas = list(_tareas.values())

    if estado:
        tareas = [t for t in tareas if t["estado"] == estado]
    if prioridad:
        tareas = [t for t in tareas if t["prioridad"] == prioridad]

    inicio = (pagina - 1) * limite
    return tareas[inicio:inicio + limite]

def obtener_por_id(tarea_id: int) -> dict | None:
    return _tareas.get(tarea_id)

```

```
def actualizar_tarea(tarea_id: int, datos: dict) -> dict | None:
    if tarea_id not in _tareas:
        return None
    _tareas[tarea_id].update({k: v for k, v in datos.items() if v is not None})
    _tareas[tarea_id]["actualizada_en"] = datetime.now()
    return _tareas[tarea_id]

def eliminar_tarea(tarea_id: int) -> bool:
    if tarea_id not in _tareas:
        return False
    del _tareas[tarea_id]
    return True
```

12.4 API principal (main.py)

python

```

# main.py
from fastapi import FastAPI, HTTPException, Query, Path, status
from typing import Optional
from modelos import (
    TareaCrear, TareaActualizar, TareaLeer,
    EstadoTarea, PrioridadTarea
)
import datos as db

app = FastAPI(
    title="API de Tareas",
    description=""
)

## Gestión de Tareas

API REST completa para gestionar tareas personales y de equipo.

### Funcionalidades
- Crear, leer, actualizar y eliminar tareas
- Filtrar por estado y prioridad
- Paginación de resultados
    "",
    version="1.0.0"
)

# ——— ENDPOINTS —————

@app.get("/", tags=["inicio"])
def inicio():
    return {
        "app": "API de Tareas",
        "version": "1.0.0",
        "documentacion": "/docs"
    }

@app.get(
    "/tareas",
    response_model=list[TareaLeer],
    tags=["tareas"],
    summary="Listar todas las tareas"
)
def listar_tareas(
    estado: Optional[EstadoTarea] = Query(
        default=None,

```



```

        description="Filtrar por estado"
    ),
    prioridad: Optional[PrioridadTarea] = Query(
        default=None,
        description="Filtrar por prioridad"
    ),
    pagina: int = Query(default=1, ge=1, description="Número de página"),
    limite: int = Query(default=10, ge=1, le=50, description="Resultados por página"
):
    """Retorna todas las tareas con soporte de filtros y paginación."""
    return db.obtener_todas(
        estado=estado,
        prioridad=prioridad,
        pagina=pagina,
        limite=limite
    )

@app.get(
    "/tareas/{tarea_id}",
    response_model=TareaLeer,
    tags=["tareas"],
    summary="Obtener tarea por ID"
)
def obtener_tarea(
    tarea_id: int = Path(..., gt=0, description="ID de la tarea")
):
    """Retorna una tarea específica por su ID."""
    tarea = db.obtener_por_id(tarea_id)
    if not tarea:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Tarea con ID {tarea_id} no encontrada"
        )
    return tarea

@app.post(
    "/tareas",
    response_model=TareaLeer,
    status_code=status.HTTP_201_CREATED,
    tags=["tareas"],
    summary="Crear nueva tarea"
)
def crear_tarea(tarea: TareaCrear):
    """Crea una nueva tarea en el sistema."""

```

```

    nueva_tarea = db.crear_tarea(tarea.model_dump())
    return nueva_tarea

@app.patch(
    "/tareas/{tarea_id}",
    response_model=TareaLeer,
    tags=["tareas"],
    summary="Actualizar tarea"
)
def actualizar_tarea(
    tarea_id: int = Path(..., gt=0),
    datos: TareaActualizar = ...
):
    """Actualiza parcialmente una tarea existente."""
    tarea = db.actualizar_tarea(tarea_id, datos.model_dump(exclude_none=True))
    if not tarea:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Tarea con ID {tarea_id} no encontrada"
        )
    return tarea

@app.delete(
    "/tareas/{tarea_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    tags=["tareas"],
    summary="Eliminar tarea"
)
def eliminar_tarea(tarea_id: int = Path(..., gt=0)):
    """Elimina permanentemente una tarea."""
    eliminada = db.eliminar_tarea(tarea_id)
    if not eliminada:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Tarea con ID {tarea_id} no encontrada"
        )

@app.get(
    "/tareas/estadisticas",
    tags=["estadísticas"],
    summary="Estadísticas de tareas"
)
def estadisticas():
    """Retorna un resumen estadístico de las tareas."""

```

```
todas = db.obtener_todas(limite=10000)

por_estado = {}
por_prioridad = {}

for tarea in todas:
    estado = tarea["estado"]
    prioridad = tarea["prioridad"]

    por_estado[estado] = por_estado.get(estado, 0) + 1
    por_prioridad[prioridad] = por_prioridad.get(prioridad, 0) + 1

return {
    "total": len(todas),
    "por_estado": por_estado,
    "por_prioridad": por_prioridad
}
```

12.5 Ejecutar y probar

bash

```
# Ejecutar
uvicorn main:app --reload

# Crear una tarea
curl -X POST http://localhost:8000/tareas \
  -H "Content-Type: application/json" \
  -d '{
    "titulo": "Aprender FastAPI",
    "descripcion": "Completar la guía completa de FastAPI",
    "prioridad": "alta"
  }'

# Listar tareas
curl http://localhost:8000/tareas

# Filtrar por prioridad
curl "http://localhost:8000/tareas?prioridad=alta&pagina=1&limite=5"

# Actualizar estado
curl -X PATCH http://localhost:8000/tareas/1 \
  -H "Content-Type: application/json" \
  -d '{"estado": "en_progreso"}'

# Eliminar
curl -X DELETE http://localhost:8000/tareas/1
```

13. Próximos pasos

Felicitaciones por completar la guía introductoria. Ahora puedes:

-  Crear endpoints REST con GET, POST, PUT, PATCH, DELETE
-  Validar datos automáticamente con Pydantic
-  Manejar parámetros de ruta, query y body
-  Documentar tu API automáticamente
-  Manejar errores correctamente
-  Estructurar proyectos básicos

Para continuar tu aprendizaje

1. Bases de datos → SQLAlchemy + Alembic con FastAPI

2. **Autenticación** → JWT tokens y OAuth2
3. **Testing** → pytest + TestClient de FastAPI
4. **Inyección de dependencias** → El sistema `Depends()` en profundidad
5. **Deploy** → Docker + servidor en la nube

Todo esto se cubre en el **Documento 2: Guía Intermedia-Avanzada de FastAPI**.

Referencias

- [Documentación oficial de FastAPI](#)
 - [Documentación de Pydantic v2](#)
 - [Starlette \(base de FastAPI\)](#)
 - [Uvicorn \(servidor ASGI\)](#)
-

Documento 1 de 2 — Guía Introductoria de FastAPI

Versión 1.0 — Actualizado para FastAPI 0.110+ y Pydantic v2